

### Question 1

✓ SUBMITTED

1 point

Which of the following inbuilt types are mutable?

☐ Numbers / Integers

☐ Strings

☒ Lists

☒ Dictionaries

☐ Tuples

### Question 2

✓ SUBMITTED

1 point

Which of the following statements are true with regard to Python and OO programming?

☒ Classes are a blueprint for creating objects in Python

☒ All variables store object references in Python

☐ You must have your user defined code within classes in Python

☒ Python supports multiple inheritance

### Question 3

✓ SUBMITTED

1 point

Which of the following concepts are required to implement Abstract Classes in Python?

☐ Import the ABC class from the abc module (Abstract Base Class)

☐ Be a subclass of ABC

☐ Have one of more abstract methods

☒ All of the above

#### Question 4

✓ SUBMITTED 1 point

I am **debugging** in my preferred IDE. I wish to step through my code without entering the code of a method to be called. Which **debugger action** should I use?

☐ Continue / Pause

☒ Step Over

☐ Step Into

☐ Step Out

#### Question 5

✓ SUBMITTED 1 point

A lot of the sample code has `if __name__ == '__main__':` at the bottom of the module. What does the statement do?

☐ It is required to run the code

☒ The code statements within this block will only be executed if Python runs this file directly

☒ It helps prevent code from being accidentally executed when the module is being imported from other file

☐ Chat GPT told me to do

#### Question 6

✓ SUBMITTED 1 point

What is the purpose of **Abstract Classes** in Python?

☐ It helps ensure any concrete subclasses must provide an implementation of certain methods

☐ It helps provide a common interface for subclasses

☐ It can help provide a partial implementation of a class

☒ All of the above

### Question 7

✓ SUBMITTED

1 point

If I were to create a new instance of the following **Cat** class, what attributes in that object would I have?

<> Python

```
1 class Animal:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def info(self):
7         return f"{self.name} is {self.age} years old"
8
9 class Dog(Animal):
10    def __init__(self, name, age, breed):
11        Animal.__init__(self, name, age)
12        self.breed = breed
13
14    def speak(self):
15        return "Woof"
16
17 class Cat(Animal):
18    def __init__(self, name, age, indoor):
19        Animal.__init__(self, name, age)
20        self.indoor = indoor
21
22    def speak(self):
23        return "Meow"
24
25 cat = Cat('Roger', 6, True)
```

- ☐ name, age
- ☐ name, age, breed
- ☒ name, age, indoor
- ☐ name, age, breed, indoor

### Question 8

✓ SUBMITTED

1 point

Why does the following code fail?

<> Python



Run

```
1 class Example:
2     def my_method():
3         pass
4
5 example = Example()
6 example.my_method()
```

- ☐ Poor naming convention
- ☐ Improper indentation
- ☒ All Python instance methods must have at least one parameter, normally called `self`
- ☐ The Ed Stem live coding tool is buggy

### Question 9

✓ SUBMITTED

1 point

By convention, where's the best place to create class variables in Python?

- ☒ At the top of the class
- ☐ In the `__init__` method
- ☐ Outside of the class
- ☐ None of the above